

Quiz-1

ELL7282/ELL782: Introduction to Computer Architecture

IIT Delhi · Semester I, 2026–27

(Answer all the questions with proper explanations.)

Total Marks: 50

Time: 60 minutes

Instructions: Answer all questions with necessary reasoning. Truth tables/K-maps are not allowed in Q1. For IEEE 754 use single precision with round-to-nearest, ties-to-even. In Q5, **sb** **a, b, L** means $a \leftarrow a - b$ and branch to L iff the new $a < 0$. SimpleRisc operations are 32-bit; default immediates are sign-extended, suffix **u** zero-extends, and suffix **h** places the immediate in bits 31:16.

1. Consider

$$F(A, B, C, D) = AB + A'C + BC + AD + A'D + CD,$$

where $A, B, C, D \in \{0, 1\}$, $+$ denotes OR, juxtaposition denotes AND, and X' denotes NOT X .

- (a) Simplify F using Boolean algebra only. [4]
- (b) Realize the simplified function using the **minimum possible number of 2-input NAND gates**. State the minimum gate count, name all intermediate signals, and justify why fewer gates are insufficient. [4]

Solution: (a) Using the consensus theorem

$$XY + X'Z + YZ = XY + X'Z,$$

with $X = A$, $Y = B$, and $Z = C$,

$$AB + A'C + BC = AB + A'C.$$

Therefore,

$$\begin{aligned} F &= AB + A'C + AD + A'D + CD \\ &= AB + A'C + D(A + A') + CD \\ &= AB + A'C + D + CD \\ &= \boxed{D + AB + A'C}. \end{aligned}$$

Here, $AD + A'D = D$ uses complementarity, and $D + CD = D$ uses the absorption law.

(b) A minimum seven-gate realization is

$$\begin{aligned} g_1 &= \text{NAND}(A, A) = A', \\ g_2 &= \text{NAND}(A, B) = (AB)', \\ g_3 &= \text{NAND}(g_1, C) = (A'C)', \\ g_4 &= \text{NAND}(D, D) = D', \\ g_5 &= \text{NAND}(g_2, g_3) = AB + A'C, \\ g_6 &= \text{NAND}(g_5, g_5) = (AB + A'C)', \\ F &= \text{NAND}(g_4, g_6) = D + AB + A'C. \end{aligned}$$

To see the lower bound, write

$$F' = D'(A' + B')(A + C').$$

The three irredundant factors require four NAND gates: one for D' , one for $(AB)' = A' + B'$, and two for A' and $(A'C)' = A + C'$. Combining three factors using only 2-input NAND gates requires three more gates. Hence fewer than seven gates cannot realize the function, and

$$\boxed{\text{minimum NAND-gate count} = 7}.$$

2. In an unknown radix r ,

$$(57.3)_r + (24.6)_r = (104.1)_r,$$

where $(X)_r$ denotes the numeral X written in base r , and the point is the radix point. Thus, for example,

$$(57.3)_r = 5r + 7 + \frac{3}{r}.$$

(a) Determine r algebraically and prove that it is the unique valid integer radix. Recall that r must exceed every digit used. [3]

(b) Let

$$N = (57.3)_r - (24.6)_r.$$

Find N in decimal. Assume a two's-complement fixed-point representation with exactly three fractional bits. Determine the minimum total word length required to represent both $+N$ and $-N$, and give the bit patterns for $+N$, $-N$, and the 16-bit sign extension of $-N$.

The binary point is assumed to lie immediately before the three least-significant bits. [3]

(c) In an n -bit two's-complement fixed-point system with f fractional bits, let $F_f(u)$ denote the unsigned integer value of the n -bit pattern representing u . The represented numerical value is therefore the signed integer associated with the pattern divided by 2^f . Prove

$$F_f(-u) \equiv 2^n - F_f(u) \pmod{2^n}.$$

Hence explain why negating the most-negative representable fixed-point number produces the same bit pattern, although the mathematical positive value is different. [3]

Solution:

(a) Expanding the radix-point numerals,

$$5r + 7 + \frac{3}{r} + 2r + 4 + \frac{6}{r} = r^2 + 4 + \frac{1}{r}.$$

Multiplying by r ,

$$7r^2 + 11r + 9 = r^3 + 4r + 1.$$

Therefore,

$$r^3 - 7r^2 - 7r - 8 = 0$$

and

$$(r - 8)(r^2 + r + 1) = 0.$$

Since $r^2 + r + 1$ has discriminant $1 - 4 = -3$, it has no real roots. Hence

$$\boxed{r = 8}.$$

The largest digit used is 7, so a valid radix must satisfy $r > 7$. Therefore, $r = 8$ is the unique valid integer radix.

(b) For $r = 8$,

$$N = (57.3)_8 - (24.6)_8 = (32.5)_8.$$

Thus,

$$N = 3(8) + 2 + \frac{5}{8} = \boxed{26.625}.$$

With three fractional bits, multiply the value by

$$2^3 = 8.$$

The corresponding scaled integer is

$$26.625(8) = 213.$$

An n -bit two's-complement code has scaled-integer range

$$-2^{n-1} \leq I \leq 2^{n-1} - 1.$$

Since 8 bits cannot represent 213 but 9 bits can,

$$\boxed{n = 9}.$$

The positive pattern is

$$213 = 011010101_2,$$

so, inserting the binary point,

$$\boxed{+N = 011010.101_2}.$$

Taking the 9-bit two's complement,

$$011010101 \longrightarrow 100101011.$$

Therefore,

$$\boxed{-N = 100101.011_2}.$$

Sign-extending to 16 bits while keeping three fractional bits gives

$$\boxed{-N = 1111111100101.011_2}.$$

Without explicitly displaying the binary point, the 16-bit pattern is

$$\boxed{1111111100101011_2 = 0xFF2B}.$$

(c) Let

$$I = 2^f u$$

be the scaled signed integer corresponding to the fixed-point value u . Its n -bit pattern satisfies

$$F_f(u) \equiv I \pmod{2^n}.$$

For $-u$, the scaled integer is $-I$. Therefore,

$$F_f(-u) \equiv -I \pmod{2^n}.$$

Since

$$-I \equiv 2^n - I \pmod{2^n},$$

we obtain

$$\boxed{F_f(-u) \equiv 2^n - F_f(u) \pmod{2^n}}.$$

The most-negative fixed-point value is

$$u_{\min} = -2^{n-f-1},$$

whose scaled integer is

$$-2^{n-1}.$$

Its bit pattern has unsigned value

$$F_f(u_{\min}) = 2^{n-1}.$$

Fixed-width negation gives

$$2^n - 2^{n-1} = 2^{n-1},$$

so the same pattern

$$\boxed{100\dots 0}$$

is produced.

This is an overflow condition. The mathematical positive value

$$+2^{n-f-1}$$

is outside the representable fixed-point range, whose maximum is

$$2^{n-f-1} - 2^{-f}.$$

3. Let $x = 2^{24}$ and $y = 1$ be IEEE 754 single-precision numbers. Here, $\text{fl}(z)$ denotes z rounded to single precision using round-to-nearest, ties-to-even.
- (a) Give the complete 32-bit hexadecimal encodings of x and y , and the numerical distance from x to the next larger representable number. [2]
- (b) Evaluate, showing each non-trivial rounding decision,

$$E_1 = \text{fl}(\text{fl}(x + y) - x), \quad E_2 = \text{fl}(x + \text{fl}(y - x)),$$

where E_1, E_2 are the final floating-point results. Explain why $E_1 \neq E_2$. [3]

- (c) Find the greatest representable number below x , its hexadecimal encoding, and explain why the representable-number gaps immediately below and above 2^{24} differ. [2]

Solution: (a)

$$x = 1.0 \times 2^{24} \implies \boxed{x = 0x4B800000},$$

and

$$y = 1.0 \times 2^0 \implies \boxed{y = 0x3F800000}.$$

At exponent 24, the spacing is

$$2^{24-23} = \boxed{2}.$$

(b) The exact sum

$$x + y = 2^{24} + 1$$

is halfway between 2^{24} and $2^{24} + 2$. Ties-to-even selects 2^{24} , so

$$\text{fl}(x + y) = x \implies \boxed{E_1 = 0}.$$

Also,

$$y - x = 1 - 2^{24} = -16777215,$$

which is exactly representable. Hence,

$$E_2 = 2^{24} - 16777215 = \boxed{1}.$$

Therefore,

$$\boxed{E_1 = 0, \quad E_2 = 1}.$$

The +1 is lost in the intermediate rounding of $x + y$ in E_1 .

(c) Immediately below 2^{24} , the spacing is

$$2^{23-23} = 1.$$

Thus the predecessor is

$$\boxed{2^{24} - 1 = 16777215}$$

with encoding

$$\boxed{0x4B7FFFFFFF}.$$

The downward gap is 1 and the upward gap is 2 because the ULP doubles when the exponent changes from 23 to 24.

4. A three-dimensional array

`short A[5][7][9]`

starts at byte address 0x8000; each element occupies 2 bytes. An element starts at 0x81BC. Here, $A[i][j][k]$ denotes the element indexed by i, j, k ; the rightmost index varies fastest in row-major order, and the leftmost varies fastest in column-major order.

- (a) Determine (i, j, k) under (i) row-major and (ii) column-major organization. Show the byte-address calculation. [4]
- (b) If the element stores decimal -300 , give its 16-bit two's-complement representation and the bytes at $0x81BC$ and $0x81BD$ for little endian (least-significant byte first) and big endian (most-significant byte first). [2]
- (c) If $r1 = 0x8000$ and $r2 = 0x1A0$, determine the addressing mode, effective address

$$EA = r1 + r2 + 28$$

(where 28 is decimal), and the corresponding row-major array element for $28[r1, r2]$. [1]

- (d) A 32-bit word $0x00613041$ is stored at $0x81C0$ on a little-endian machine. List its bytes in increasing-address order and give the NUL-terminated ASCII string, where NUL is $0x00$. Are the printable-character byte values identical in UTF-8? [1]

Solution: The byte displacement and element offset are

$$0x81BC - 0x8000 = 0x1BC = 444, \quad 444/2 = 222.$$

(a) For row-major order,

$$222 = 63i + 9j + k.$$

Therefore,

$$i = 3, \quad j = 3, \quad k = 6,$$

and the element is

$$A[3][3][6].$$

For column-major order,

$$222 = i + 5j + 35k.$$

Therefore,

$$k = 6, \quad j = 2, \quad i = 2,$$

and the element is

$$A[2][2][6].$$

(b) The 16-bit representation is

$$-300 = 111111011010100_2 = 0xFED4.$$

Little endian:

$$0x81BC:D4, \quad 0x81BD:FE.$$

Big endian:

$$0x81BC:FE, \quad 0x81BD:D4.$$

(c) The mode is **base-index-offset**. Since $28 = 0x1C$,

$$EA = 0x8000 + 0x1A0 + 0x1C = 0x81BC.$$

Thus the row-major element is

$$A[3][3][6].$$

(d) The increasing-address byte sequence is

$$41 \ 30 \ 61 \ 00,$$

which represents the NUL-terminated string

$$"A0a".$$

Yes. ASCII characters use identical single-byte values in UTF-8.

5. The machine has only

sbn a,b,L : $a \leftarrow a - b$; branch to L iff the new $a < 0$,

where a, b are memory operands and L is a label. Initially,

$$x \geq 0, \quad y > 0, \quad q = 0, \quad t = 0, \quad \text{one} = 1,$$

where x_0 denotes the initial value of x , q is the quotient, t is temporary storage, and **one** stores constant 1.

(a) Write an SBN-only program terminating with

$$q = \left\lfloor \frac{x_0}{y} \right\rfloor, \quad x = x_0 \bmod y.$$

Preserve y and assume no other initialized constants.

[6]

(b) State a loop invariant, meaning a relation true before every iteration, and give a termination argument proving the final values of q and x . [2]

Solution: (a) A valid program is

```

loop:      sbn x, y, restore
           sbn t, t, makeMinusOne

makeMinusOne:
           sbn t, one, increment

increment:
           sbn q, t, clearTemp

clearTemp:
           sbn t, t, jumpBack

jumpBack:
           sbn t, one, loop

restore:
           sbn t, t, makeMinusY

makeMinusY:
           sbn t, y, undo

undo:      sbn x, t, done

done:
```

A successful subtraction leaves $x \geq 0$, after which $t = -1$ and

$$q \leftarrow q - (-1) = q + 1.$$

When $x - y < 0$, the restore path forms $t = -y$ and performs

$$x \leftarrow x - (-y) = x + y,$$

undoing the failed subtraction. The value of y is never modified.

(b) At the start of every iteration,

$$\boxed{x_0 = qy + x, \quad x \geq 0}.$$

Initially this holds because $q = 0$ and $x = x_0$. After a successful iteration,

$$(q + 1)y + (x - y) = qy + x = x_0,$$

so the invariant is preserved.

The loop exits when the next subtraction gives $x - y < 0$, so before that trial,

$$0 \leq x < y.$$

After restoration,

$$x_0 = qy + x, \quad 0 \leq x < y,$$

which implies

$$q = \lfloor x_0/y \rfloor, \quad x = x_0 \bmod y.$$

Each successful iteration reduces x by $y > 0$, so only finitely many iterations are possible.

6. (a) A processor has fixed 32-bit instructions, register-to-register ALU operations, explicit load/store memory access, and one unified byte-addressed memory. Here, ALU means arithmetic logic unit; RISC/CISC describe ISA styles; Harvard/Von Neumann describe memory organization. Classify the ISA and memory organization, and explain briefly how a large register file exploits locality. [2]
- (b) Register **r0** contains a 32-bit word x , **r9** contains a destination byte address, and **0[r9]** denotes memory at address $r9 + 0$. Write a straight-line SimpleRisc program, with no loop or function call, that reverses

$$B_3B_2B_1B_0 \longrightarrow B_0B_1B_2B_3,$$

leaves the result in **r1**, stores it at **0[r9]**, and preserves **r0,r9**. Use at least one immediate modifier: **u** means zero extension and **h** places the immediate in bits 31:16.

For $x = 0x12345678$, state the final **r1**. Also explain why logical right shift, which inserts zeros, is required instead of arithmetic right shift, which replicates the sign bit. [8]

Solution: (a) The fixed-length instructions, register-to-register operations, and load/store organization indicate a

RISC-like ISA.

A single memory for instructions and data gives a

Von Neumann organization.

A large register file retains recently reused operands near the ALU, exploiting temporal locality and reducing memory accesses.

(b) A valid program is

```
movh r3, 0xFF00
addu r3, r3, 0xFF00    // r3 = 0xFF00FF00
not  r2, r3            // r2 = 0x00FF00FF

and  r4, r0, r2
lsl  r4, r4, 8
and  r5, r0, r3
lsr  r5, r5, 8
or   r6, r4, r5

lsl  r7, r6, 16
lsr  r8, r6, 16
or   r1, r7, r8
st   r1, 0[r9]
```

The masks select alternate bytes. The first pair of shifts swaps adjacent bytes, and the second pair swaps the two 16-bit halfwords. For

$$r0 = 0x12345678,$$

$r6 = 0x34127856$

and finally

$r1 = 0x78563412$.

The `u` in `addu` zero-extends `0xFF00` to `0x0000FF00`. A logical right shift is required because it inserts zeros. An arithmetic right shift could replicate bit 31 and introduce unwanted 1s into the extracted byte fields.

— End of Answered Question Paper —